

nginx -> Envoy Gateway Migration Lab

Automated rollback + annotation translation at 100-endpoint scale

Reza Farshi — reza.farshi@gmail.com

June 2026

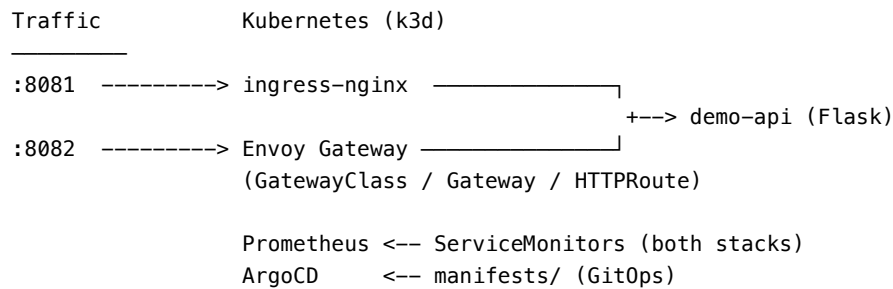
Problem

Migrating 100 nginx Ingresses to Envoy Gateway introduces two hard problems:

1. **You cannot watch a dashboard per endpoint** — rollback must be automated, metric-driven, and fleet-wide
2. **Each Ingress has different annotations** — timeout, retry, rate-limit, auth, CORS — all need translation to Gateway API CRDs without manual per-route work

This lab implements both.

Architecture



Both stacks serve the **same backend simultaneously**. Migration = shifting backendRefs weights in the HTTPRoute. No DNS changes required during canary.

Stack: k3d · Envoy Gateway v1.2 · ingress-nginx · kube-prometheus-stack · ArgoCD · Gateway API

Automated Rollback (fleet-wide, no human in the loop)

The key constraint: at 100 endpoints a human cannot watch per-route dashboards. Rollback must fire automatically from metrics.

How it works

Every canary HTTPRoute is labeled migration=canary. A CronJob (canary-watchdog) queries Prometheus every 60s:

```
envoy 5xx / total > 5%          -> rollback
p99 downstream latency > 2000ms -> rollback
```

On breach, one loop hits all 100 routes atomically — ~5 to 10 seconds end-to-end:

```
kubectl get httproute -A -l migration=canary \
-o custom-columns="NS:.metadata.namespace,NAME:.metadata.name" --no-headers \
```

```
| while read -r ns name; do
  kubectl delete httproute "$name" -n "$ns" --ignore-not-found
done
```

Three rollback levels (defence-in-depth)

Level	Action	Speed	When
L1	Patch weights: nginx=100 envoy=0	< 2s	Soft breach, keep HTTPRoute
L2	Delete HTTPRoute	< 5s	Hard breach, xDS pushes removal
L3	Delete Gateway entirely	< 5s	Total failure, nginx handles 100%

Files

File	Purpose
manifests/rollback/cronjob.yaml	CronJob: Prometheus query -> threshold check -> rollback
manifests/rollback/rbac.yaml	ServiceAccount + ClusterRole (patch/delete HTTPRoutes cluster-wide)
manifests/rollback/prometheus-rule.yaml	PrometheusRule alerts (error rate + latency breach)
scripts/auto-rollback.sh	Rollback logic: --delete or --reweight, supports DRY_RUN=true

```
make rollback-install  # deploy CronJob + RBAC + PrometheusRule
make rollback-test     # manual trigger (DRY_RUN=true to preview)
make canary-status     # show weight split across all canary HTTPRoutes
```

Annotation Translation (100-endpoint problem)

Each nginx Ingress carries different annotations. Manual translation per route does not scale.

scripts/translate-annotations.sh automates the full Phase-0 audit:

1. Inventories every annotation across all Ingresses (`kubectl get ing -A -o json | jq`)
2. Flags hard cases with no direct mapping (server-snippet, modsecurity-snippet)
3. Calls ingress2gateway (SIG tool) to generate HTTPRoute manifests
4. Emits per-Ingress BackendTrafficPolicy + SecurityPolicy stubs -> manifests/generated/

Annotation -> Envoy Gateway mapping

nginx annotation	Envoy Gateway CRD	Field
proxy-read-timeout	BackendTrafficPolicy	timeout.http.requestTimeout
proxy-connect-timeout	BackendTrafficPolicy	timeout.tcp.connectTimeout
proxy-next-upstream-tries	BackendTrafficPolicy	retry.numRetries
limit-rps	BackendTrafficPolicy	rateLimit.local
rewrite-target	HTTPRoute filter	URLRewrite.path
ssl-redirect	HTTPRoute filter	RequestRedirect.scheme: https
whitelist-source-range	SecurityPolicy	ipAllowList.cidrRanges
auth-url	SecurityPolicy	extAuth.http
affinity: cookie	BackendTrafficPolicy	loadBalancer.consistentHash

nginx annotation	Envoy Gateway CRD	Field
server-snippet / modsecurity	EnvoyExtensionPolicy	Wasm filter — manual work

Each policy attaches per-route via targetRef so different annotations stay independent across 100 routes.

```
make annotation-audit      # fleet-wide annotation inventory
make translate-annotations # generate all stubs -> manifests/generated/
```

Observability

Prometheus ServiceMonitor on both stacks. Grafana dashboard for **demo visibility** — not the operational monitoring mechanism. The CronJob is the operational mechanism.

Live failure injection for demo/testing:

```
make chaos      # 10% random 500s on demo-api
make bad-traffic # hammer /fail -> error rate spikes -> watchdog fires
make slow-traffic # hammer /slow?seconds=2 -> p99 breach -> watchdog fires
```

Demo Flow

```
make up          # bootstrap full cluster (~4 min on MacBook)
make demo-reset  # nginx only, no Envoy
make demo-step2  # deploy Envoy parallel (zero nginx downtime)
make demo-step5  # show weighted canary split in HTTPRoute
make chaos       # inject failures -> watch CronJob auto-rollback fire
make demo-step6  # manual rollback demonstration
```

github.com/farshi/nginx-to-envoy-lab